Troll Farm Bot, from Zero

Learn the Rust in the code — then understand every layer of the algorithm

This is a beginner-first manual for the exact readable program currently submitted on the CodinGame Troll Farm arena as agent `6593838`, submission `41089629`.

Subject: `e7a-without-orchard-readable.rs`
Exact SHA-256:
`18f379024c8366ca90469be684e11d0a4362fd7c8e932ecac90e2a0be6565cf4`
Size: 88,593 bytes · 1,921 physical lines · 1,916 code lines

No Rust knowledge is assumed. Start at Chapter 1 and read in order; experienced programmers can jump to Part III, “The algorithm.”

How to use this manual

The program is short enough to fit on CodinGame, but dense enough to look hostile to a new reader. This manual unfolds it in layers. First you learn the game vocabulary. Then you learn only the Rust features the program actually uses. Finally, you trace a full turn from standard input to the semicolon-separated command line sent to the referee.

Every source reference has the form **L453–474**, meaning lines 453 through 474 of the exact readable source named on the title page. “The bot” always means that exact source. Historical features—especially the removed secure apple-orchard wrapper—are clearly marked as absent.

The one-minute explanation

The application is a deterministic two-worker heuristic controller:

1. Read the map once, then read a complete game snapshot every turn.
2. During the opening, collect the missing PLUM, LEMON, APPLE, and IRON needed to train the strongest affordable second troll quickly.
3. For each troll, generate possible jobs: bank carried goods, gather opening resources, chop a tree, plant banked fruit for later conversion, harvest only as a fallback, or wait.
4. Give every job a numeric score. Normal tree work is scored mainly as **wood delivered per turn**.

5. With two trolls, examine every compatible pair of jobs and choose the pair with the largest combined score.
6. Rewrite colliding moves so the two friendly trolls do not land on the same cell.
7. Print the commands, remember a small amount of intent, and repeat next turn.

```
map + turn snapshot
        ↓
GameState
        ↓
opening/endgame decision + candidates per troll
        ↓
exact two-troll pair maximization
        ↓
door and movement conflict repair
        ↓
MSG / TRAIN / MOVE / CHOP / PICK / PLANT / DROP / MINE / WAIT
```

This is not machine learning and not search through future game trees. It is a carefully layered greedy scheduler with short analytic forecasts for tree growth, chopping, travel, and banking.

Part I — The world the program controls

1. What Troll Farm is

Two players control trolls on a small grid. Grass cells are walkable. Water, iron, rocks, and both shacks are not walkable. Trees occupy grass cells, so a troll can stand on a tree. Each turn both players issue commands, the referee applies them in a fixed priority order, trees grow, and scores are recomputed.

The bot calls our side player `0` and the opponent player `1`. A "shack door" means a walkable grass cell orthogonally adjacent to a shack; trolls deposit resources from a door because the shack cell itself is not walkable.

1.1 Resources and points

Item	Constant	Main role in this bot	Banked score
PLUM	`0`	movement-speed training cost; possible planting seed	1
LEMON	`1`	carry-capacity training cost; denial focus	1
APPLE	`2`	harvest-power training cost; possible planting seed	1
BANANA	`3`	planting seed; not used in training	1
IRON	`4`	chop-power training cost	0
WOOD	`5`	main scoring resource	4

Only resources in the shack inventory score. Cargo in a troll's hands is not yet score.

1.2 Troll skills

Each troll has four integer skills:

- `movement_speed`: how many BFS steps a MOVE may cover in one turn;
- `carry_capacity`: total units of all cargo it can carry;
- `harvest_power`: fruit units it may take per HARVEST;
- `chop_power`: tree health removed per CHOP, and iron mined per MINE up to free capacity.

The starting troll is capable of harvesting. Every second-troll option generated by this program sets `harvest_power` to zero and varies movement, carrying, and chopping from 1 through 3.

1.3 Tree life cycle

Trees are PLUM, LEMON, APPLE, or BANANA. They grow from size 0 to 4. At size 4 they begin producing fruit, up to three stored fruit. Water next to a tree shortens its cooldown. Chopping removes health; when the tree dies, its current size becomes the amount of wood available to the choppers.

Kind	Base cooldown	Water reduction	Health formula
PLUM	8	5	`4 + 2 × size`
LEMON	8	5	`4 + 2 × size`
APPLE	9	7	`8 + 3 × size`
BANANA	6	2	`2 + 1 × size`

1.4 Referee order matters

The referee resolves tasks as MOVE → HARVEST → PLANT → CHOP → PICK → TRAIN → DROP → MINE. This is why the original troll can MOVE away from the shack on the same turn that `TRAIN` creates the second troll: textual command order is not execution order.

2. What this exact version does—and does not do

This source contains the Yamo/Moisan two-troll controller plus opening, regeneration, doorway, and idle-harvest refinements. It was derived from Yann Moisan's documented third-place contest idea: train two trolls, score tree work by delivered wood throughput, coordinate their targets, and use remaining fruit in the endgame.

It does **not** contain the 375-line secure apple-orchard wrapper removed for the code-cost experiment. It also never trains a third troll, never learns from data, never rolls out complete future games, and never physically blocks an enemy unit—enemy and friendly units may share a cell.

Part II — Rust, taught through this program

3. Reading the file without fear

The source is a mechanically expanded version of a compact CodinGame submission: a generator adds whitespace back to the exact token stream, so the layout is close to idiomatic Rust but is produced by rule rather than by `rustfmt`. Arithmetic, borrows and generics stay tight (`n+ms*ms`, `&mut`), and very long expressions wrap at commas, method-chain dots or boolean operators. The whole file has three large pieces:

Lines	Module	Responsibility
L6–377	`game`	data types, rules, pathfinding, and text input
L378–1896	`bot`	candidate generation, scoring, coordination, persistent policy state
L1897–1921	crate root / `main`	buffered input/output and the turn loop

4. Values, variables, and basic types

Rust infers most local types:

```
let mut turn = 1;
turn += 1;
```

`let` creates a binding. Bindings are immutable unless marked `mut`. Common types in the bot are `i32` for game integers, `usize` for array/vector indices, `f64` for candidate scores, `bool`, `String`, and `&str`.

The annotation `let travel: i32` would force a type, but the compiler usually infers it from the called function and arithmetic. Casts such as `values[1] as usize` are explicit because Rust does not silently convert integer types.

5. Ownership and borrowing: `T`, `&T`, and `&mut T`

The most important Rust distinction in this file is:

- `GameState` means an owned value;
- `&GameState` means a shared, read-only borrow;
- `&mut self` means the method may change the bot's persistent memory;
- `&mut [String]` means the collision resolver may rewrite commands in place.

For example, candidate generation receives `view: &GameState`; it can inspect the snapshot but cannot alter it. `commands(&mut self, view: &GameState)` may update `type_to_cut`, training intent, and regeneration commitments while treating the referee snapshot as immutable.

6. Arrays, vectors, tuples, and ordered collections

The alias `type Stock = [i32; ITEM_COUNT]` makes a six-element fixed array. A `Vec<T>` is a growable sequence. A tuple such as `(i32, i32)` represents a cell. `BTreeSet<Cell>` and `BTreeMap<K,V>` keep keys sorted, making iteration and tie behavior deterministic.

Rust form	Meaning here
`[i32; 6]`	six resource counts with fixed indices
`Vec<Unit>`	all current trolls, count known only at runtime
`(i32, i32)`	`(x, y)` grid coordinate
`BTreeSet<Cell>`	sorted unique walkable/iron/water cells
`BTreeMap<i32, Vec<Candidate>>`	candidate list keyed by troll id
`VecDeque<Cell>`	FIFO queue for breadth-first search

7. Structs, enums, methods, and traits

A `struct` groups named fields. `Unit` combines identity, owner, position, skills, and cargo. An `enum` is one value chosen from named alternatives. `PlantKind` is one of four species; `Target` describes the reservation attached to a candidate.

`impl Unit { ... }` defines methods. `impl Bot for YamoBot` implements a trait—an interface saying that every bot must provide `commands`. Associated functions such as `MoisanBot::ceil_div` do not receive `self`; methods such as `ensure_opening(&mut self, ...)` do.

`#[derive(Clone, Copy, Debug, Eq, PartialEq, ...)]` asks the compiler to generate routine behavior. `Copy` makes small values like coordinates and enums copy automatically. `Clone` permits explicit duplication of owned data such as command strings.

8. `Option`: representing “maybe” safely

Rust uses `Option<T>` instead of null:

```
enum Option<T> { Some(T), None }
```

The parser's `PlantKind::parse` returns `Some(kind)` for a valid word and `None` otherwise. `read_static_map` uses `?`: if any required token is missing or invalid, the entire function returns `None`. `let Some(predicted) = ... else { continue; };` skips a tree that will die before arrival. `if let Some(cell) = detour` handles the successful detour case.

9. Pattern matching and control flow

`match` must cover every case. It converts plant names to enum values, plant kinds to constants, and target variants to cells. `if`, `else if`, `for`, `while let`, and early `return` are used normally. The `else` attached to `let ... else` is Rust's concise “require this shape or leave.”

10. Iterators and closures

Code such as

```
view.units.iter()
    .filter(|unit| unit.player == 0)
    .count()
```

borrows each unit, keeps ours, and counts them. Iterator chains are lazy until a consumer such as `count`, `sum`, `min`, `max_by_key`, or `collect` runs them. A closure `|unit| ...` is an anonymous function. Double references like `**cell` arise when an iterator is borrowing an already borrowed collection element—not because there are two cells.

11. Strings, parsing, and formatting

Input arrives as `String` lines. `split_whitespace()` produces borrowed `&str` tokens. `parse().ok()?` converts a token and propagates failure as `None`. Commands are owned `String` values built with `format!`, for example `format!("MOVE {} {} {}", id, x, y)`.

The final `commands.join(";")` creates the exact semicolon-separated line expected by CodinGame. `writeln!` adds the newline and `flush()` sends it immediately instead of leaving it buffered.

Part III — The algorithm

12. The complete turn pipeline

The main policy entry is `YamoBot::commands` at L1793. Its order is the application architecture:

1. remove completed or stale regeneration commitments;
2. initialize the permanent tree-type focus and desired second-troll specification;
3. downgrade or abandon the opening after the training deadline if necessary;
4. decide whether to emit `TRAIN` now;
5. classify the turn as opening, normal, or endgame;
6. build one candidate list per own troll;
7. add special idle-harvest or shack-egress choices when applicable;
8. repair a one-door traffic jam at the candidate level;
9. choose the best compatible candidate pair;
10. repair projected movement collisions;
11. remember any selected `PICK` as a regeneration commitment;
12. prepend first-turn `MSG` and current-turn `TRAIN`, then return the commands.

The remaining chapters expand every step.

13. Input becomes a `GameState`

The application is not connected to a game engine through a library. CodinGame starts it as a process, writes text to standard input, and reads one line of text from standard output every turn. The `game::protocol` module at L241–375 is the adapter between that text and typed Rust values.

13.1 Static input, read once

`read_static_map` reads `width height` and then exactly `height` rows (L265–275). Each character is interpreted by `parse_static_map` (L276–301):

Character	Meaning	Stored where
`.`	walkable grass	`walkable`
`0`	our shack	`shacks[0]`
`1`	opponent shack	`shacks[1]`
`+`	iron deposit	`iron`
`~`	water	`water`
anything else	rock or irrelevant	nowhere

The shack is deliberately not inserted into `walkable`. This looks dangerous because a troll is born on the shack, but the BFS routine seeds its source even when that source is not walkable; it can therefore expand from the shack to a grass neighbor.

13.2 Dynamic input, read every turn

`read_turn` at L302–374 reads, in order:

1. two inventory lines, each with six integers;
2. a tree count and seven fields per tree;
3. a troll count and fourteen fields per troll.

The exact tree record is:

```
type x y size health fruits cooldown
```

The exact troll record is:

```
id player x y movementSpeed carryCapacity harvestPower chopPower
carryPlum carryLemon carryApple carryBanana carryIron carryWood
```

Any missing line, invalid number, wrong field count, or unknown tree kind returns `None`. In `main`, `while let Some(view) = read_turn(...)` then ends the process cleanly. That is robust against EOF, but it also means malformed referee input would terminate the bot rather than produce a diagnostic.

13.3 The snapshot data dictionary

Type/field	Lines	Meaning	Used by policy?
`Cell = (i32, i32)`	L9	`(x,y)` coordinate	everywhere
`Stock = [i32; 6]`	L17	fixed resource vector	opening, scoring, banking
`PlantKind`	L18–48	four fruit/tree species	parsing and all tree logic
`Stats`	L49–60	four troll skills	training and action timing
`Unit`	L61–72	id, owner, cell, stats, cargo	candidate generation
`Plant`	L73–81	species, cell, growth and health state	forecasts and targets
`GameState.walkable`	L86	all grass cells	BFS and legal destinations
`GameState.shacks`	L87	our and enemy bases	banking and denial geometry
`GameState.inventories`	L88	banked resources	affordability and score
`GameState.scores`	L91	derived bank scores	endgame trigger
`GameState.turn`	L92	1-based turn number	deadlines and time budget
`GameState.next_id`	L93	max seen id plus one	parsed but unused by policy
`GameState.iron/water`	L94–95	static terrain sets	mining and tree cooldown

`GameState::plant_at` and `GameState::unit` are small linear searches (L97–104). They return `Option`, forcing callers to handle “not found.”

13.4 Score is recomputed locally

`rules::score` at L155–157 calculates

```
PLUM + LEMON + APPLE + BANANA + 4 × WOOD
```

It reads only shack inventory. Iron, carried items, troll stats, and standing trees are worth zero at that moment. `read_turn` computes both players' scores from the two inventories rather than trusting a separate score field.

14. Navigation: BFS, speeds, and waypoints

Manhattan distance is cheap geometry: $|x_1-x_2| + |y_1-y_2|$ (L176–178). It ignores rocks and water. Real travel uses breadth-first search over `walkable` cells (L185–204).

14.1 Breadth-first search in plain language

`bfs_distances` puts all source cells in a FIFO queue at distance zero. It repeatedly removes the oldest cell, visits its four orthogonal neighbors, and gives every previously unseen walkable neighbor distance `d+1`. On an unweighted grid, the first discovered distance is shortest.

If `V` is the number of walkable cells, one BFS costs `O(V)` time and `O(V)` memory because each cell has at most four edges.

14.2 `next_cell` predicts one MOVE

The command `MOVE id tx ty` names a final target, but a troll may move only its speed this turn. `next_cell` at L205–239 predicts the landing cell:

1. BFS from the current cell.
2. If the target is reachable within `speed`, land exactly on it.
3. If the target is reachable but farther away, BFS backward from the target and choose an in-range cell with the smallest remaining distance.
4. If the target is unreachable—most notably a shack—find reachable cells with minimum Manhattan distance to it and route toward that goal set.
5. Break equal choices lexicographically by coordinate.

That last tie rule is deterministic and useful for testing, but it is not perfect referee parity: the official referee uses continuing random state for equal-best movement cells.

14.3 Distances versus turns

A BFS distance counts cells traveled. Turns are usually

```
ceil(distance / movement_speed)
```

implemented by `ceil_div` at L475–481. A distance of 5 takes 5 turns at speed 1, 3 turns at speed 2, and 2 turns at speed 3. The function returns the sentinel `10_000` if the divisor is invalid.

15. Memory: what survives between turns

`GameState` is replaced every turn. `YamoBot`, constructed once at L1913, stores the policy memory. The full field list is at L431–447.

Field	Active initial value	Purpose
`announced`	`false`	emit the identifying `MSG` once
`announcement`	carry/regen/transit name	first-turn message text
`type_to_cut`	`None`	permanent PLUM/LEMON denial focus chosen once
`desired_second`	`None`	chosen second-troll stats plus estimated ETA
`opening_initialized`	`false`	prevent repeated initial selection
`opening_abandoned`	`false`	stop opening collection if deadline fallback fails
`opening_policy`	`TUNED_CARRY`	constants controlling training choice
`idle_regeneration`	`true`	when no chop exists, allow fruit-to-tree conversion work
`persistent_regeneration`	`true`	remember a chosen seed until its conversion job completes
`door_unblocking`	`true`	run the unique-door traffic coordinator
`partial_bank_transit`	`true`	let partly loaded carriers use that coordinator
`idle_harvest`	`true`	permit truly idle harvest-capable fallback
`idle_harvest_clock_only`	`false`	fallback may run in either endgame trigger, not only t>250
`regeneration_commitments`	empty map	troll id → species selected by `PICK`
`opponent_eta_penalty`	`0`	opponent-arrival risk adjustment is compiled but inactive

The constructor first creates a feature-off base at L972–990, then `tuned_carry_regeneration_transit_idle_harvest` enables the five refinements at L991–1000. Reading both functions is essential: a field present in the struct is not necessarily active.

16. Opening: choose and fund the second troll

The bot is architecturally capped at two trolls. `can_train` returns false when `n >= 2` before checking money (L517–528). The opening's only purpose is therefore to create troll number two.

16.1 Training cost

With `n` existing trolls and requested skills `(ms, cc, hp, chop)`, L146–154 computes:

```
PLUM  = n + ms²
LEMON = n + cc²
APPLE = n + hp²
IRON  = n + chop²
```

For the common request `(2,3,0,3)` while one troll exists, the bill is PLUM 5, LEMON 10, APPLE 1, IRON 10. BANANA and WOOD are never training currency.

Every generated second-troll option has `hp=0`. Thus APPLE costs only `n=1`, normally covered by the random starting inventory. Opening ETA deliberately estimates PLUM, LEMON, and—if the map has iron—IRON, but not APPLE (L1044–1057).

16.2 Twenty-seven possible specifications

`opening_options` at L1087–1105 enumerates:

```
movement_speed = 1..3
carry_capacity = 1..3
harvest_power  = 0
chop_power     = 1..3
```

That is `3 × 3 × 3 = 27` options. Policy clamps prevent accidental values outside 1–3.

16.3 Crude time-to-bill estimator

`collection_eta` at L1010–1043 starts distances from our shack doors. For each missing unit:

- iron estimate = `missing × (2 × distance + 2)` to the nearest reachable cell beside iron;
- fruit estimate = the same round-trip term plus any ripening wait at the best matching tree.

This is intentionally simple. It does not pack several units into one high-capacity trip and does not simulate competition. It is a ranking estimate, not an exact future schedule.

`ticks_until_fruit` at L529–550 simulates cooldown and growth for at most 100 turns. Existing fruit has wait zero. Otherwise the tree grows until size 4 and then reaches its next fruit-production tick, using water-adjusted cooldown.

16.4 Select the strongest quick option

`choose_second_troll` at L1106–1157 first finds options with estimated ETA at most 15. It ranks them by total `ms+cc+chop`, then lower ETA, then—under the active policy—higher chop, carry, movement. If no option meets the horizon it falls back to `(1,1,0,1)`.

The active `TUNED_CARRY` policy prefers at least carry 2 and chop 1. If the baseline lacks those, it accepts a preferred option only when it costs no more than 15 extra estimated turns and still fits the turn-35 deadline. `prefer_movement_ties=false`, so chop wins the final skill tie before carry and movement.

16.5 Opening resource actions

Before troll two exists, `early_candidates` at L551–582 does this for the original troll:

1. always include WAIT;
2. if carrying anything or full, route to a shack door and DROP;
3. otherwise compute the desired bill;
4. for each missing PLUM/LEMON/APPLE, add matching fruit-tree actions;
5. for missing IRON, add mine/approach actions;
6. if no resource action was generated, fall back to ordinary chopping.

`fruit_candidates` prefers HARVEST on the current ripe tree at `base+900`; every other living matching tree gets a MOVE scored `base - travel - wait` (L583–612). `iron_candidates` similarly scores adjacent MINE at `base+900` and approach moves at `base-distance` (L613–638).

16.6 Training now, deadline downgrade, or abandonment

`training_affordable` checks exact bank inventory and the two-troll cap (L1158–1168). At or after turn 35, `enforce_training_deadline` acts if the chosen spec is still unaffordable (L1180–1215):

- if the policy required a preferred class, switch to the strongest affordable preferred option;
- under the active non-required policy, switch to the strongest affordable option of any class;
- if none is affordable, mark the opening abandoned.

`can_train` adds one more guard: it refuses training in the final 20 turns. When training is legal, the driver emits `TRAIN ms cc hp chop` and ensures a troll currently on the shack also receives a MOVE option so MOVE resolves before TRAIN (L1797–1809 and L1860–1874).

17. Candidate architecture: command, score, reservation

A `Candidate` at L397–400 has exactly three fields:

```
struct Candidate {
    command: String,
    score: f64,
    target: Target,
}
```

The command is executable text. The score permits comparison across job types. The target is not sent to the referee; it is an internal reservation used to prevent two friendly trolls from being assigned the same place.

`Target` variants at L393–396 distinguish no reservation, the abstract shack, a particular bank door, a general cell, and a tree cell. Most incompatibility ultimately reduces to “same cell.”

17.1 The priority bands

The scores are deliberately separated into broad bands:

Approximate score	Meaning
`20_000`	forced doorway/traffic action
`10_000`	continue chopping the tree already under the troll in endgame
`9_000`	plant a carried fruit immediately when conversion is feasible
`8_000`	DROP now or move toward an endgame planting cell
`7_500`	select a regeneration seed at the shack
`7_000`	return cargo toward a shack door; urgent late conversion PICK
`6_500`	vacate the shack to make TRAIN legal
`6_100` / `6_000`	opening iron / fruit collection
dynamic below ~ `1_000`	wood throughput, early conversion efficiency, idle harvest
`0`	WAIT

These bands encode a lexicographic policy with ordinary floating-point comparison. A forced traffic action wins over planting; planting wins over banking travel; opening collection wins over normal tree throughput; WAIT loses whenever useful work has positive score.

17.2 Banking cargo

`MoisanBot::bank_candidates` at L488–516 BFSes from the troll to every walkable shack door.

- Already at the door: `DROP id`, score 8000.
- Else: `MOVE id doorX doorY`, score `7000 - travelTurns`.
- If there is no reachable door candidate: MOVE toward the unwalkable shack, relying on `next_cell`'s unreachable-target behavior to park nearby.

The Yamo wrapper at L1221–1229 removes a bank door currently occupied by another friendly troll, unless the action is not a move to that occupied door.

18. Tree prediction and wood-throughput scoring

Normal play is built around one calculation:

```
score = 1000 × collectibleWood
              / (travelTurns + chopTurns + returnTurns + 1 DROP turn)
```

The factor 1000 only places the ratio in the desired score band. The objective is delivered wood per turn, not the largest tree and not the nearest tree in isolation.

18.1 Predict enemy damage before arrival

`predicted_opp_chop` at L639–654 sums chop power of enemy trolls currently on the tree. If none is present but health is below the expected full health for that kind and size, it assumes one point of enemy chop per future turn. Otherwise it predicts zero.

This is a small opponent model. It does not forecast enemy movement, target changes, or harvests.

18.2 Simulate the tree until our arrival

`predict_tree` at L655–686 loops once per travel turn:

1. subtract predicted enemy chop and reject the tree if health reaches zero;
2. decrement cooldown;
3. on cooldown zero, grow size and health if below size 4;
4. otherwise generate fruit if below three;
5. reset cooldown after growth/fruit.

Only size, health, and cooldown are returned because the chopping calculation does not value fruit.

18.3 Simulate our chopping

`chop_outcome` at L687–716 repeatedly subtracts our chop power. If the tree survives a turn and its cooldown reaches zero while size is below four, it grows and gains the species-specific health slope. The result is `(chopTurns, finalSize)`. A tree that cannot be killed within 100 simulated turns is rejected.

The simulation omits continued opponent damage after our arrival. That damage would often make the tree die sooner, so the forecast is not a full joint combat simulator.

18.4 Build each chop candidate

`chop_candidates` at L717–782 rejects a troll with no chop power or no free capacity. For every living reachable tree it then:

1. computes travel turns by BFS distance and movement speed;
2. rejects trees predicted dead before arrival;
3. computes shortest return time to any shack door;
4. simulates chopping and final tree size;
5. adds one turn for DROP;
6. rejects jobs that cannot finish before the 300-turn limit;
7. caps wood by the troll's free capacity;
8. scores delivered wood per total turn;
9. emits CHOP if already on the tree, otherwise MOVE toward its cell.

18.5 Worked throughput comparison

Suppose troll A has speed 2, chop 2, and two free slots.

- Tree X: 4 travel cells → 2 turns; 3 chop turns; 3 return turns; final size 2.
- Tree Y: 2 travel cells → 1 turn; 5 chop turns; 2 return turns; final size 3 but only 2 fit.

Including one DROP turn:

```
X = 1000 × 2 / (2 + 3 + 3 + 1) = 222.22
Y = 1000 × 2 / (1 + 5 + 2 + 1) = 222.22
```

They tie despite different distance, health, and size. Deterministic iteration order then decides which equal-scored candidate survives later selection.

19. PLUM/LEMON focus and denial bonus

`focus_type` at L453–474 runs once. It BFSes from our shack doors and sums distances to all LEMON trees and all PLUM trees. An unreachable tree contributes the sentinel 10,000.

The exact branch is subtler than "choose the closer species":

```
if lemon <= plum and plum - lemon <= 8: choose PLUM
else if lemon <= plum:                  choose LEMON
else:                                   choose PLUM
```

Therefore LEMON is selected only when its total distance is more than eight better. A tie or small LEMON advantage still selects PLUM.

For a focus-kind tree, while the opponent has at most two trolls, L768–771 adds:

```
900 / (1 + ManhattanDistance(tree, opponentShack))
```

So a focus tree on the enemy door gets +900; distance 2 gets +300; distance 8 gets +100. This rewards removing training-currency trees near the enemy. It does not explicitly estimate whether the enemy can harvest and replant the fruit before removal.

`yamo_chop_candidates` at L1476–1517 can additionally subtract risk when an enemy chopper reaches a tree no later than us. In this exact live constructor `opponent_eta_penalty=0`, so the function returns immediately and that adjustment is inactive.

20. Two-troll coordination

Independent greedy choices would often send both trolls to the same tree or door. `select` at L811–865 treats the pair as one tiny optimization problem.

20.1 Spatial compatibility

`compatible` at L788–799 permits any pair involving `Target::None`. Otherwise two targets are incompatible when they refer to the same cell. This keeps two trolls off the same tree, general cell, or shack door. The abstract `Target::Shack` also conflicts with another identical abstract shack target.

20.2 Inventory compatibility

`picked_item` recognizes `PICK id ITEM` text. If both candidates PICK the same resource, they are compatible only when at least two units exist in the shack inventory (L800–810). This is a small transactional reservation: two trolls cannot both spend the last single seed.

20.3 Exact pair maximization

With exactly two troll ids, the bot examines the Cartesian product of their candidate lists:

```
for every candidate a for troll 1
    for every candidate b for troll 2
        if targets and stock are compatible
            evaluate a.score + b.score
```

The highest sum wins. If candidate lists have C_1 and C_2 members, cost is $O(C_1 \times C_2)$. The code uses strict `score > best_score`; a numerical tie retains the first compatible pair encountered. Because ids and map structures are `BTree` collections, this tie behavior is deterministic.

The source contains a greedy fallback for more than two trolls, but `can_train` makes that branch unreachable in normal play. It remains defensive generic code, not a scaling strategy.

20.4 Worked pair choice

Troll 1 candidate	Score	Target	Troll 2 candidate	Score	Target	Pair
Tree A	300	A	Tree A	450	A	forbidden
Tree A	300	A	Tree B	260	B	**560**
Tree C	240	C	Tree A	450	A	**690, winner**
Bank door	6997	D	Tree A	450	A	7447 if distinct

High-priority banking naturally dominates tree work, while target compatibility prevents both trolls from claiming the same physical resource.

21. Movement conflict repair

Candidate compatibility prevents shared final targets, but two different destinations can still produce the same one-turn landing cell. `resolve_move_conflicts_with_priority_and_forbidden` at L886–970 repairs the actual projected moves.

21.1 Projection

For every MOVE, the bot finds the unit, final target, and predicted landing via `next_cell`. Units that do not move reserve their current cells. A MOVE whose landing equals its current cell becomes WAIT immediately.

21.2 Winner order

Movers are sorted by priority membership and then descending unit id. The normal driver supplies an empty priority set, so the higher id reserves its desired landing first. This mirrors the referee's within-player highest-id collision advantage, though the bot is repairing commands before sending them rather than relying on a blocked action.

21.3 Detour or wait

If the desired landing is already reserved, the lower-priority mover considers four adjacent walkable cells. It rejects cells reserved for the current output, occupied at turn start, or marked forbidden. Among the rest it chooses the cell with best distance to the original target, breaking ties by coordinate. If no

detour exists, it emits WAIT.

Finally, even successful MOVE text is rewritten to the predicted **one-turn landing cell**, not the original far destination. The policy replans from a fresh snapshot next turn.

> This collision layer has no cross-turn route memory. Replanning plus deterministic equal-choice rules can create period-2 movement oscillation. The project measured and separately studied that limitation; this source keeps the original behavior.

22. The unique-door traffic coordinator

Some maps have exactly one walkable cell beside our shack. With two trolls, that door is both an exit for a newly trained troll and the only DROP location. `force_unique_door_clear` at L1266–1441 rewrites candidate lists before pair selection.

It first does nothing unless `unique_shack_door` finds exactly one door. The remaining branches handle these concrete situations:

Situation	Repair
blocker on door carries noncommitted cargo; another troll is on shack	force blocker DROP, force transit WAIT
blocker has a safe planned egress	force blocker to its projected landing, transit to door
no planned egress	choose a free neighbor as blocker egress
a carrier's next landing is occupied by an empty troll	preserve a valid egress, swap, or clear one step
door blocker carries ordinary cargo	force DROP and possibly hold incoming carrier
door blocker and carrier can swap	force the two opposite moves
no swap	choose a free egress biased away from the incoming carrier

“Committed” fruit is treated specially: a troll carrying a seed selected for regeneration should not DROP it merely to clear traffic. The function therefore consults `carries_committed_fruit` before forced banking.

This routine is not general multi-agent path planning. It is a targeted two-troll state machine for the high-value bottleneck immediately around a one-door shack.

23. Normal mode, regeneration, and persistent intent

`main_candidates` at L1518–1574 is the ordinary post-training scheduler. It always starts with WAIT, then applies a sequence of guards whose order is the policy:

1. If safe regeneration is active and the troll carries fruit, return only banking candidates.
2. If it carries anything and is already beside the shack, add DROP/bank candidates.
3. Under a sparse-map condition, add high-priority seed PICK candidates.
4. If cargo is full, return banking candidates immediately.

5. Generate normal chop candidates.
6. If no chop exists and idle regeneration is enabled, switch to `endgame_candidates` even when the global endgame trigger is false.
7. If no chop exists but cargo exists, bank it; otherwise append the chops.

The sparse-map seed condition at L1534–1550 requires all of:

- safe/persistent regeneration active;
- empty cargo;
- turn at least 100;
- at most two plants left on the entire map;
- both own trolls already exist;
- the troll is beside our shack on a cell without a tree.

It adds `PICK` in inventory priority BANANA, PLUM, LEMON, APPLE with scores just below 7500.

23.1 Why a selected seed needs memory

Without memory, the next turn's greedy ranking might tell a seed-carrying troll to bank the fruit again or start unrelated work. `remember_selected_regeneration` at L1459–1475 parses every selected `PICK id KIND` and records `id → kind`.

On later turns `reconcile_regeneration_commitments` at L1442–1458 keeps that entry while any of these is true:

- the troll still carries the chosen fruit;
- the troll carries WOOD, meaning the planted tree is being converted and returned;
- the troll stands on a living tree of that chosen species.

Otherwise the entry is removed. A committed troll is routed directly through `endgame_candidates` by L1819–1828, which supplies the sequence plant → chop → bank. This is a small persistent job state, not a general planner.

24. Endgame and fruit-to-wood conversion

`endgame` at L1787–1790 is true when either:

```
turn > 250
OR
(total plant count <= 4 AND our bank score < opponent bank score)
```

The second trigger begins recovery early when the forest is nearly exhausted and we are behind. The first trigger is unconditional, so late play always enters this policy even while leading.

24.1 If the troll carries fruit

`endgame_candidates` at L1617–1675 scans every empty reachable grass cell not occupied by another friendly troll. For each it estimates whether the complete conversion can finish before turn 300:

```
travel to cell + 1 PLANT + chop planted tree + return to door + 1 DROP
```

`conversion_chop_turns` at L1588–1616 simulates a newly planted size-1 tree while it is being chopped. The tree may grow and gain health during chopping. If the full transaction fits:

- already on the chosen cell → `PLANT`, score 9000;
- elsewhere → MOVE toward it, score `8000 - BFS distance`.

If any feasible planting candidate exists, the function returns it immediately with WAIT; normal trees are not considered that turn. If no conversion fits, it banks the fruit instead.

24.2 If the troll carries nonfruit cargo

Any remaining cargo—normally WOOD or IRON—causes an immediate return of banking candidates (L1676–1679). The job is not allowed to drift into another tree target before depositing.

24.3 If the troll is already chopping

With empty cargo, the function generates normal chop candidates. If one command is `CHOP id` on the current cell, its score is raised to 10,000 and returned immediately (L1680–1692). Continuing an in-progress tree beats leaving to start conversion elsewhere.

24.4 Pick a banked seed for conversion

With empty cargo and no current CHOP, the bot considers banked fruit in this fixed order:

```
BANANA → PLUM → LEMON → APPLE
```

BANANA is first because its low health and short growth cycle generally make it cheap to convert. If already at a shack door on an empty cell and the planted tree can be chopped and banked in time, the candidate is `PICK id KIND`. Otherwise it considers moving toward each usable shack door and checks the same time budget there.

After turn 250, the scores are urgent bands near 7000 for PICK and 6000 for approach. Before turn 251—when this routine was entered because no chop exists or the forest is sparse and we trail—the score is an efficiency ratio:

```
750 / (travel + conversion chop + 3 fixed action turns)
```

Finally, ordinary chop candidates are appended. The bands decide whether conversion or remaining natural tree work wins.

24.5 This is not the removed secure orchard

This exact source can still plant fruit. What was removed is the larger secure apple-orchard wrapper: dedicated site geometry, protected mother-tree selection, activation economics, camping, and maintenance coordination. The remaining planting is base-policy regeneration/conversion, usually aimed at turning one banked point into size-dependent WOOD before time expires.

25. Idle harvesting: a deliberately weak fallback

`idle_harvest_candidates` at L1752–1786 is available only to a troll with empty cargo and positive harvest power. In this bot that is normally the original troll; the trained troll has harvest zero.

A candidate tree must be alive, ripe, reachable, returnable, and finish harvest plus bank before turn 300. If an empty enemy troll is already on the tree, the candidate is skipped unless our troll is also already there. Score is only `1 / completeTripTurns`.

The driver adds these candidates only in endgame and only when the existing candidate list contains nothing except Target::None/WAIT (L1855–1859). This makes harvesting a work-conservation fallback, not a rival to chopping or conversion. It cannot implement a renewable fruit economy by itself.

26. Exact driver truth table

The per-troll routing at L1818–1854 is easiest to understand as a priority table. The first matching row wins:

Condition	Candidate generator
troll has a persistent regeneration commitment	`endgame_candidates`
global endgame, persistent mode, troll currently carries fruit	`main_candidates` with safe regeneration; this banks/protects fruit
global endgame	`endgame_candidates`
opening active, fewer than two trolls, not training now	`early_candidates`
otherwise	`main_candidates`

The second row seems surprising: a fruit carrier in global endgame is sent to safe normal logic, which immediately offers banking and returns. A committed fruit carrier uses the first row and is allowed to plant. This distinction prevents uncommitted fruit from being casually converted.

After candidate generation:

- idle harvest may be appended to a WAIT-only endgame list;
- if TRAIN happens now, `PICK` candidates are removed so training currency is not spent earlier in the referee's PICK→TRAIN order;
- a troll still on the shack gets a 6500 MOVE egress candidate;
- unique-door repair may replace whole lists with forced 20,000-point actions;
- exact pair selection chooses commands;
- movement repair rewrites landings;
- selected `PICK` commands become commitments.

26.1 Whole-turn pseudocode

```
commands(view):
    forget finished regeneration jobs
    if first policy turn:
        choose permanent PLUM/LEMON focus
        choose desired second-troll stats
    if training deadline passed:
        downgrade to an affordable spec or abandon opening

    train_now = opening active AND exact desired bill legal
    output MSG once
    output TRAIN if train_now

    phase = opening / normal / endgame
    for each own troll in ascending id:
        candidates[id] = generator selected by truth table
        maybe add idle HARVEST
        protect TRAIN inventory and shack egress

    if the shack has exactly one door:
        repair door traffic in candidate space
    selected = highest-score compatible pair
    repair projected same-cell MOVE landings
    remember selected seed PICKs
    append selected unit commands
    if absolutely nothing exists, output WAIT
```

27. Output protocol and command lifecycle

The `Bot` trait at L1893–1895 requires one method returning `Vec<String>`. `main` at L1905–1921 constructs buffered stdin/stdout locks, reads the static map, constructs the active Yamo bot, and loops from turn 1 until input ends.

Commands are joined with semicolons:

```
MSG yamo-carry-regen-transit-idle-harvest-rust;MOVE 0 4 7;TRAIN 2 3 0 3
```

The exact order in the vector does not control referee priority. Every line is flushed immediately because an interactive judge waits for the answer before producing the next snapshot.

The source can emit these commands:

Command	Origin in policy
`MSG text`	once, first call
`TRAIN ms cc hp chop`	opening bill affordable
`MOVE id x y`	almost every job's approach/return/egress
`HARVEST id`	opening collection or idle fallback
`CHOP id`	on selected tree
`PICK id KIND`	take banked seed for regeneration/conversion
`PLANT id KIND`	place carried seed on current empty cell
`DROP id`	deposit all cargo at a shack door
`MINE id`	collect iron beside a deposit during opening
`WAIT`	no work or deliberate traffic hold

28. Why the algorithm is effective despite being greedy

The policy has no neural network and no full-game rollout, but it is not "choose nearest tree." It gets strength from five forms of structure:

1. **Whole-job pricing.** Tree score charges travel, changing health/size, chopping, return, and the DROP turn before comparing targets.
2. **Resource-aware capacity.** Collectible wood is capped by free cargo, so a huge tree does not attract a nearly full troll for unusable yield.
3. **Small exact coordination.** With two workers, exhaustive pair selection is cheap and removes a large class of duplicate assignments.
4. **Priority separation.** Banking and opening bills occupy score bands well above routine work, encoding economic deadlines without a general optimizer.
5. **Minimal persistent state.** The permanent denial focus, desired training specification, and regeneration commitment prevent several kinds of one-turn indecision.

Its success also depends on the architecture staying small. The exact-pair optimizer and special one-door coordinator are tractable because there are at most two trolls. Simply lifting the roster cap would not create a coherent three-worker economy.

Part IV — Worked turns

29. Walkthrough A: opening resource collection

Assume the one-time opening selector chose second-troll stats `(ms=2, cc=2, hp=0, chop=2)`. With one existing troll the bill is:

```
PLUM  = 1 + 2² = 5
LEMON = 1 + 2² = 5
APPLE = 1 + 0² = 1
IRON  = 1 + 2² = 5
```

Suppose our bank is `[4, 5, 3, 7, 5, 0]`. Only one PLUM is missing. The starter has empty cargo and free capacity.

1. `commands` finds `train_now=false` because PLUM 4 < 5.
2. Own roster size is one, so `early=true`.
3. `early_candidates` starts with WAIT.
4. It loops over the bill. LEMON, APPLE, and IRON already pass. PLUM is deficient.
5. `fruit_candidates(PLUM, base=6000)` runs.
6. If the starter stands on a ripe PLUM, HARVEST scores 6900. Otherwise every living PLUM gets `6000 - travelTurns - ripeningWait`.
7. Pair selection has only one troll, so it takes the maximum-scored candidate.

After HARVEST, the fruit is cargo, not bank inventory. Next turn `early_candidates` sees `carrying_any=true`, returns banking candidates, and sends the starter to a door. At the door it emits DROP. Only after DROP resolves does the next snapshot show PLUM 5.

This explains a common beginner confusion: “I harvested the missing resource, why didn't TRAIN happen immediately?” HARVEST fills troll cargo. TRAIN reads the shack inventory, and DROP has lower referee priority than TRAIN, so the bill becomes usable on the following turn.

30. Walkthrough B: TRAIN and shack evacuation on the same turn

Now assume the exact bill is already banked and the original troll still stands on the unwalkable shack spawn cell.

1. `can_train` sees `n=1`, more than 20 turns remain, and all four bill inequalities pass.
2. The driver appends `TRAIN 2 2 0 2`.
3. During candidate generation, it notices `train_now` and the troll on `shacks[0]`.
4. If no normal candidate is a MOVE, it adds a MOVE to the first walkable orthogonal neighbor at score 6500.
5. Output contains both TRAIN and MOVE.
6. The referee applies MOVE before TRAIN, vacating the shack.
7. TRAIN then rechecks shack occupancy, spends the bank bill, and creates troll two.

The source might print TRAIN before MOVE in the semicolon line; referee task priority still makes MOVE happen first. This is why understanding the external execution model matters more than text order.

31. Walkthrough C: two trolls choose different trees

Suppose after forecasts the candidate lists are:

```
troll 0: WAIT 0, tree A 310, tree B 270
troll 1: WAIT 0, tree A 420, tree C 250
```

Naive independent maxima choose tree A for both. The exact pair loop evaluates:

- A+A: rejected, same `Target::Tree(A)`;
- A+C: 310+250 = 560;
- B+A: 270+420 = 690;
- B+C: 270+250 = 520;
- all WAIT combinations: legal but lower.

The selected jobs are troll 0 → B and troll 1 → A. This is globally better than giving A to troll 0 and forcing troll 1 onto C. The optimizer does not need a sophisticated solver because there are only two lists.

Now suppose the shortest first steps toward B and A land on the same intermediate cell. Candidate targets are different, so the pair was spatially compatible. The movement repair projects both landings, lets the higher-id troll reserve the cell, and gives the other a best adjacent detour or WAIT. Pair selection and collision repair solve different layers of the problem.

32. Walkthrough D: chop, return, and score

A troll with carry capacity 3 and empty cargo selects a size-4 APPLE. It is 4 BFS cells away, the troll has speed 2, predicted chop time is 7 turns, and the closest return door is 5 cells away.

```
travel = ceil(4/2) = 2 turns
return = ceil(5/2) = 3 turns
drop   = 1 turn
wood   = min(final size 4, free capacity 3) = 3
total  = 2 + 7 + 3 + 1 = 13 turns
score  = 1000 × 3 / 13 = 230.77
```

The first command is MOVE toward the tree. Every later turn the bot recomputes the world rather than blindly following a stored route. Once on the tree, the candidate command is CHOP. When death gives the troll wood and fills capacity, `main_candidates` immediately returns bank candidates. At the door, DROP transfers three WOOD into inventory, and local bank score rises by 12.

If another tree becomes better during travel, the bot may retarget because ordinary chop jobs have no persistent commitment. That responsiveness is useful, but with deterministic path ties it can also contribute to oscillation.

33. Walkthrough E: a one-door traffic jam

Imagine a map with one door `D`:

- troll 0 stands on `D` with one WOOD;
- troll 1 is on the shack after training and needs to exit.

`force_unique_door_clear` matches "blocker at door + transit at shack." Because troll 0 has cargo and it is not a committed regeneration seed, the function replaces troll 0's entire list with `DROP 0` at score 20,000 and troll 1's list with WAIT. The pair selector cannot override this.

On the next snapshot troll 0's cargo is empty. The coordinator can force it to a safe egress and move troll 1 onto `D`. This costs a turn for the new troll but avoids two conflicting moves and preserves the banked WOOD.

If troll 0 were carrying a committed seed, forced DROP would destroy the plant-convert job. The coordinator instead searches for egress/swap behavior because `carries_committed_fruit` protects that cargo.

34. Walkthrough F: regeneration transaction

Assume it is turn 180, both trolls exist, only two trees remain, and troll 0 is empty at a shack door. Bank inventory has BANANA.

1. The sparse-map condition in `main_candidates` adds `PICK 0 BANANA` near score 7500.
2. If selected, `remember_selected_regeneration` records `0 → BANANA` immediately after command selection.
3. Next snapshot, troll 0 carries BANANA; the commitment survives reconciliation.
4. The driver routes troll 0 through `endgame_candidates` even if the global endgame condition is false.
5. It finds an empty reachable cell where PLANT, chopping the new BANANA, return, and DROP all fit.
6. It MOVE(s) there, then emits PLANT at score 9000.
7. Standing on the new live BANANA keeps the commitment alive.
8. The current-tree CHOP gets score 10,000, so conversion continues.
9. After tree death, troll 0 carries WOOD; that also keeps the commitment alive while it returns.
10. After DROP, the next snapshot has neither chosen fruit, WOOD cargo, nor the matching tree under the troll. Reconciliation deletes the entry.

This is the closest thing in the source to a multi-turn job. It is implemented by one map entry and three observable completion conditions rather than a large enum of explicit phases.

Part V — Rust close-reading clinic

35. Expressions return values

Rust blocks can be expressions. In `effective_cooldown` (L139–145), the final expression has no semicolon and becomes the return value:

```
plant_cooldown(kind) - if near_water {
    water_boost(kind)
} else {
    0
}
```

The `if` itself returns either an integer boost or zero. In contrast, a semicolon discards an expression's value and makes the block return `()`—the unit type.

36. `?`, `let-else`, and early exits

The parser is a compact demonstration of failure propagation:

```
let header = read_line(reader)?;
let width = parts.next()?.parse().ok()?;
```

Every `?` means "if this is None, return None from the current function; otherwise unwrap the value." It replaces several nested `match` statements.

In a loop, the source often uses:

```
let Some(predicted) = predict_tree(...) else {
    continue;
};
```

This means the happy path requires `Some`; a dead-before-arrival tree skips to the next iteration.

37. Borrowing through iterator layers

Consider L1632–1643, which iterates a `BTreeSet<Cell>` and then filters it. `iter()` yields `&Cell`. `filter` receives a reference to that iterator item, so its closure argument can become `&&Cell`; hence `**cell` when passing an owned `(i32,i32)` to `plant_at`. The compiler is tracking references precisely. It is not copying the map or allocating extra cells.

Useful iterator endings in this source include:

Ending	Result
`.count()`	number of matching items
`.sum::<i32>()`	arithmetic total with stated type
`.collect::<Vec<_>>()`	materialize sequence as vector
`.min()` / `.max()`	best value by natural ordering
`.min_by_key(f)`	item whose computed key is smallest
`.find(f)`	first matching item as `Option`
`.any(f)` / `.all(f)`	Boolean quantifier

38. Safe defaults and deliberate panics

The code uses several ways to handle missing data:

- `unwrap_or(10_000)`: pessimistic sentinel for unreachable distance;
- `unwrap_or_else(Self::wait)`: generate a safe WAIT when no candidate fits;
- `?`: propagate parse or lookup absence;
- `unreachable!()`: assert that prior logic makes a match arm impossible;
- `expect("write command line")`: panic if stdout fails, because interactive play cannot recover.

`max_by(|a,b| a.score.total_cmp(&b.score))` uses `total_cmp` rather than partial float comparison. It defines an order even for unusual values such as NaN. The formulas here should not produce NaN, but total ordering keeps the selection code simple and explicit.

39. Strings as an internal protocol

The bot stores executable commands as `String` early, then later reparses some of them:

- `picked_item` splits `PICK id ITEM` to reserve inventory;
- `move_command` splits `MOVE id x y` to project and rewrite movement;
- `remember_selected_regeneration` parses selected PICK commands.

This is compact for a submission, but a larger application would usually store a typed `enum Command` and serialize only at the output boundary. Typed commands would eliminate repeated string parsing and make impossible commands harder to construct.

40. Why `BTreeMap` instead of `HashMap`

Hash maps have no stable iteration order. Here ordering affects equal-score ties and which worker moves first. `BTreeMap` / `BTreeSet` sort their keys, so the same snapshot produces the same candidate iteration and output. Determinism is valuable for replay testing even when a hash map might be slightly faster.

41. Generics without ceremony

`read_line(reader: &mut impl BufRead)` accepts any mutable type implementing `BufRead`: a locked stdin reader in production or an in-memory cursor in a test. This is static polymorphism—the compiler generates appropriate code without a runtime interface object.

The `Bot` trait is another abstraction. `impl Bot for YamoBot` provides the concrete method. `main` imports the trait so the method is available. There is no dynamic dispatch here because the concrete `YamoBot` type is known.

42. There is no hidden concurrency or unsafe code

The program is single-threaded, synchronous, and deterministic given snapshots plus its own memory. It uses no `unsafe`, no external crates, no files, no network, no random number generator, and no wall clock. Standard library collections, I/O, arithmetic, and string formatting are enough.

Part VI — Build, run, observe, and change it safely

43. Compile the exact source

The file is a complete one-file Rust crate. From the repository worktree:

```
rustc --edition=2021 -O \
  local_codex_1/readable-orchard-code-cost/e7a-without-orchard-readable.rs \
  -o /tmp/troll-farm-readable-bot
```

`--edition=2021` selects the language edition used by constructs such as array `into_iter`. `-O` enables optimization, matching deployment intent. The output path is outside the repository because the executable is a reproducible build artifact, not research data.

Check empty-input behavior:

```
/tmp/troll-farm-readable-bot </dev/null
echo $?
```

It should print nothing and exit with status 0 because `read_static_map` sees EOF and `main` returns.

44. Run it with a transcript

The program needs one static map followed by complete turn snapshots. A hand-written toy input is easy to get wrong because every inventory, tree, and troll field is positional. Prefer a sanitized captured transcript or a project fixture.

```
/tmp/troll-farm-readable-bot < input.txt > commands.txt 2> debug.txt
```

There must be exactly one output line per complete turn. The first normally includes `MSG`; later lines do not. Split a line on semicolons to inspect individual commands.

> Never print debugging text to stdout: CodinGame parses stdout as commands, so a friendly “selected tree A” message can become an unknown command and lose the game. Use `eprintln!`, which writes stderr, or the legal `MSG` command when you intentionally want visible game text.

45. Verify you are studying the same application

Before comparing behavior, hash the source:

```
sha256sum \
  local_codex_1/readable-orchard-code-cost/e7a-without-orchard-readable.rs
```

Expected:

```
18f379024c8366ca90469be684e11d0a4362fd7c8e932ecac90e2a0be6565cf4
```

The manual builder performs the same check and refuses to render against a different file:

```
python3 local_codex_1/readable-no-orchard-manual/build_manual.py
```

It also extracts the symbol inventory into `source-index.json`. This prevents a manual from quietly drifting to a similarly named but behaviorally different bot.

46. A practical debugging method

When a move looks wrong in a replay, do not begin by changing a weight. Trace these layers in order:

1. **Snapshot:** Are inventory, cargo, tree health/size/cooldown, positions, and turn parsed as you think?
2. **Phase:** Was the troll opening, normal, endgame, or regeneration-committed?
3. **Candidate set:** Which commands existed? A missing action is a generation/filter issue, not a scoring issue.
4. **Score:** What numeric score did each surviving candidate receive?
5. **Pair filter:** Was the best single candidate rejected because the other troll reserved its target or the same last seed?
6. **Door rewrite:** Did unique-door logic replace the candidate list?
7. **Move rewrite:** Did projected landing conflict turn a MOVE into a detour or WAIT?
8. **Referee:** Did task priority, occupancy, affordability, or a rule reject the emitted command?

This sequence mirrors the program. Jumping straight from replay symptom to one scoring constant often edits the wrong layer.

46.1 Useful temporary diagnostics

In a learning copy, print to stderr:

```
eprintln!(
    "turn={} unit={} cmd={} score={} target={:?}",
    view.turn, unit.id, candidate.command, candidate.score, candidate.target
);
```

`Target` derives `Debug`, so `{:?}` works. Candidate does not derive `Debug`, but its fields can be printed individually. Remove or gate verbose output before deployment; logging can consume the 50 ms turn budget.

47. How to make a safe learning copy

The exact live/readable artifact is hash-locked evidence. Do not edit it in place if you want comparisons to remain meaningful. Copy it to a new, clearly named learning file, then experiment.

Good first exercises:

1. Replace the first-turn announcement string and confirm only the first output line changes.
2. Add a unit test for `training_cost(1, (2,3,0,3)) == [5,10,1,0,10,0]`.
3. Instrument `focus_type` and build a tiny map where LEMON wins only after exceeding the eight-step PLUM preference margin.
4. Create two candidate lists and test that same-tree pairs are rejected.
5. Construct a movement diamond and inspect the lexicographic equal-path landing.

Do not start by changing several systems at once. A source that trains different stats, changes tree score, and rewrites collisions simultaneously cannot teach you which change caused an observed result.

48. Testing levels

Level	Question answered	Example
arithmetic unit test	Is a formula implemented correctly?	training bill, cooldown, ceil division
component fixture	Does one layer choose the expected candidate?	tree forecast, pair compatibility
transcript equality	Does a source reproduce commands on recorded snapshots?	exact command stream
closed-loop simulation	Do changed actions create stable future states?	full games on valid referee model
live health	Does it compile and emit legal commands on platform?	runtime/identity checkpoint
live value	Does mature score improve under controlled measurement?	serialized Arena experiment

Teacher-forced transcript equality is powerful for refactors, but insufficient for strategy changes: once one command differs, the future snapshots that the new bot would create are no longer the recorded snapshots.

Part VII — Limitations and design consequences

49. What the bot cannot reason about

49.1 It cannot scale beyond two trolls

`can_train` and `training_affordable` reject `n>=2`. The candidate selector has a generic greedy branch for more workers, but the live policy cannot reach it. Removing one condition would not solve workforce funding, production, mining, or multi-worker task allocation.

49.2 The trained troll cannot harvest

All 27 opening specs hard-code `harvest_power: 0`. The second troll is a mover/carrier/chopper. This is why the idle-harvest fallback mostly belongs to the starter and why planted fruit is usually converted into WOOD rather than farmed as a renewable resource.

49.3 Opponent modeling is narrow

The bot sees enemy count, position, skills, and current tree contact. It predicts current/inferred enemy chop while we travel and uses enemy-shack distance for denial. It does not predict enemy training bills, harvest plans, planting, target switches, or future movement. The compiled opponent-arrival penalty is disabled at zero.

49.4 The tree forecast is local, not a referee clone

It simulates growth and damage needed for target valuation. It does not simulate simultaneous actions, last-wood duplication, continued opponent chop after arrival, or alternative enemy commands. Its output is a useful job estimate, not an exact counterfactual future.

49.5 Ordinary jobs have no commitment

Only regeneration PICKs create persistent memory. A troll moving toward a natural tree may change its mind next turn. This gives responsiveness but can induce target reversals and movement oscillation when small score or tie differences flip repeatedly.

49.6 Denial is a geometry bonus, not an economic proof

The `900/(1+distance)` term does not calculate remaining focus resources, enemy harvest throughput, time to clear the species, or whether the enemy can replant. It can therefore spend turns on denial that is strategically ineffective on a rich or already renewable map.

49.7 Friendly movement repair is myopic

The resolver protects only current landings. It has no path reservation across turns. A detour chosen now can be reversed next turn. Enemy bodies are correctly ignored because cross-player physical blocking does not exist.

49.8 Local path ties differ from the official referee

The source selects lexicographically among equal-best next cells. The referee uses randomness from its continuing RNG state. Most decisions remain sensible, but exact local/replay parity requires proving ties absent or modeling the referee RNG.

49.9 Malformed input ends silently

Protocol functions collapse errors into `Option::None`; the turn loop exits. That is compact for a trusted judge but less diagnosable than a production parser with structured error messages.

49.10 Score bands hide trade-offs

A 7000 banking move beats any 300-point chop even if banking one low-value item costs a long detour. This is intentional policy priority, not a common-unit value function. Changing band boundaries can alter behavior discontinuously.

50. Time and space complexity

Let V be walkable cells, T trees, U own trolls (at most 2), and C_i candidates for troll i.

- One BFS: $O(V)$ time and memory.
- Tree candidate generation: several BFS calls plus up to 100 forecast iterations per tree, approximately $O(V + 100T)$ per troll.
- Two-troll selection: $O(C_1 \times C_2)$.
- Movement repair: $O(U \times V)$ in the worst small-grid view because detour ranking can BFS.
- Persistent storage: map/snapshot collections $O(V+T+allUnits)$ plus a tiny commitment map.

The implementation repeats BFS rather than caching all distance maps. On the small contest boards and two-worker roster this is comfortably simple. On a large map or many-worker architecture, distance caching and typed job structures would matter.

51. Architectural character

The algorithm is best described as a **two-agent, throughput-priced, rule-priority scheduler**:

- *two-agent*: exact coordination relies on only two workers;
- *throughput-priced*: normal economic work is valued by delivered wood per total job time;
- *rule-priority*: opening, banking, conversion, and traffic use separated score bands and guards;
- *scheduler*: it chooses current jobs, not a long action sequence;
- *stateful at narrow boundaries*: permanent opening/focus decisions and seed commitments survive.

This label is more accurate than “greedy bot.” Greedy describes the immediate selection horizon, but not the whole-job cost model, exact pair optimization, traffic state machine, and commitment mechanism that shape the choices.

52. Where the orchard went

The exact deployed readable source is the physically stripped variant. Compared with the same readable parent that included the secure apple-orchard layer:

Form	Code lines	Compact characters
exact E7a with orchard	2,502	62,820
activation disabled, code retained	2,495	62,581
physically stripped, this manual's subject	1,916	47,807

The removed 586 readable lines were 25 lines of orchard types/state, 402 helper/strategy lines, 117 per-turn wrapper lines, and 42 reservation/import/main-wiring lines. Generic APPLE parsing, harvesting, chopping, cargo, planting, and denial remain because the base policy uses them.

Live measurements found blanket orchard removal weaker in an earlier run. This manual describes the requested stripped deployment; it does not claim removal is the strongest strategy.

Part VIII — Source map and glossary

53. Call graph at a glance

```
main
 ├─ read_static_map → parse_static_map
 ├─ YamoBot::tuned_carry_regeneration_transit_idle_harvest
 └─ per turn: read_turn
       └─ YamoBot::commands
           ├─ reconcile_regeneration_commitments
           ├─ ensure_opening
           │   ├─ MoisanBot::focus_type → bfs_distances
           │   └─ choose_second_troll
           │       ├─ opening_options → opening_objective
           │       └─ collection_eta → ticks_until_fruit / bfs_distances
           ├─ enforce_training_deadline → strongest_affordable
           ├─ per unit candidate generator
           │   ├─ early_candidates → fruit_candidates / iron_candidates /
bank_candidates
           │   ├─ main_candidates → yamo_chop_candidates / bank_candidates
           │   └─ endgame_candidates → conversion_chop_turns / chop candidates
           ├─ force_unique_door_clear
           ├─ MoisanBot::select → compatible / stock_compatible
           ├─ resolve_move_conflicts → next_cell / bfs_distances
           └─ remember_selected_regeneration
```

The tree-work subgraph is:

```
yamo_chop_candidates
 └─ MoisanBot::chop_candidates
     ├─ bfs_distances (travel and return)
     ├─ predict_tree
     │   └─ predicted_opp_chop + cooldown/health rules
     ├─ chop_outcome
     └─ throughput score + optional focus denial bonus
```

54. Exhaustive function and type index

54.1 Core types, rules, navigation, and protocol

Symbol	Start	Purpose
`Cell`, `Stock`, item constants	L9	coordinate and six-resource representation
`PlantKind::{parse,as_str,item_index}`	L19	typed species and protocol/resource conversion
`Stats::tuple`	L50	four-skill record and training-cost tuple
`Unit::{total_carried,free_capacity}`	L62	troll snapshot and cargo helpers
`Plant`	L74	dynamic tree snapshot
`GameState::{plant_at,unit}`	L83	full turn snapshot and lookup helpers
`plant_cooldown`, `water_boost`	L112	species timing constants
`tree_health_params`, `tree_health`	L128	species health model
`effective_cooldown`	L139	water-adjusted cooldown
`training_cost`	L146	nonlinear bill formula
`score`, `rules::item_index`	L155	bank scoring and command-item parsing
`manhattan`, `ortho_neighbors`, `is_adjacent`	L176	elementary grid geometry
`bfs_distances`	L185	exact walkable shortest paths
`next_cell`	L205	one-turn landing prediction
`StaticMap`	L249	immutable map data
`read_line`	L257	CR/LF-safe line read
`read_static_map`, `parse_static_map`	L265	initialization parser
`read_turn`	L302	dynamic snapshot parser

54.2 Candidate and opening system

Symbol	Start	Purpose
`Target`, `Candidate`	L394	executable option plus reservation key
`YamoOpeningPolicy::TUNED_CARRY`	L403	opening thresholds and tie preferences
`OpeningObjective`	L428	stats plus estimated funding ETA
`YamoBot`	L431	persistent controller memory
`PredictedTree`	L449	arrival-time forecast subset
`focus_type`	L453	permanent PLUM/LEMON focus choice
`ceil_div`, `carry_total`, `carrying_any`	L475	arithmetic/cargo helpers
`MoisanBot::bank_candidates`	L488	DROP/return choices
`can_train`	L517	cap, time, and affordability gate
`ticks_until_fruit`	L529	next-ripe estimator
`early_candidates`	L551	opening phase dispatcher
`fruit_candidates`	L583	HARVEST/approach for a missing fruit
`iron_candidates`	L613	MINE/approach for missing iron
`collection_eta`	L1010	approximate bill-acquisition cost
`opening_objective`, `opening_key`	L1044	evaluate/rank one training spec
`opening_options`	L1087	enumerate 27 specs
`choose_second_troll`	L1106	horizon/preference constrained choice
`training_affordable`, `strongest_affordable`	L1158	exact fallback candidates
`enforce_training_deadline`	L1180	turn-35 downgrade/abandonment
`fallback_second_troll`	L1216	minimum `(1,1,0,1)` spec

54.3 Tree work, coordination, and movement

Symbol	Start	Purpose
`predicted_opp_chop`	L639	current/inferred enemy tree damage
`predict_tree`	L655	simulate tree until our arrival
`chop_outcome`	L687	simulate our chopping and growth
`chop_candidates`	L717	full-job throughput candidates
`wait`	L783	zero-score safe candidate
`compatible`	L788	spatial reservation rule
`picked_item`, `stock_compatible`	L800	same-seed inventory reservation
`select`	L811	one-worker max, exact pair max, generic greedy fallback
`move_command`	L866	parse MOVE text
`resolve_move_conflicts*`	L873	project, reserve, detour, or wait
`YamoBot::bank_candidates`	L1221	occupancy-filtered bank choices
`unique_shack_door`, `forced_move`	L1230	traffic primitives
`carries_committed_fruit`, `planned_egress`	L1244	protect seed jobs and inspect exits
`force_unique_door_clear`	L1266	two-troll doorway state machine
`yamo_chop_candidates`	L1476	optional opponent-arrival penalty wrapper

54.4 Regeneration, endgame, and application entry

Symbol	Start	Purpose
`reconcile_regeneration_commitments`	L1442	end completed/stale persistent seed jobs
`remember_selected_regeneration`	L1459	create commitment from selected PICK
`main_candidates`	L1518	ordinary post-opening scheduler
`carried_fruit`, `inventory_fruits`	L1575	fruit detection and fixed seed order
`conversion_chop_turns`	L1588	planted-tree chop forecast
`endgame_candidates`	L1617	plant/chop/bank/remaining-tree scheduler
`idle_harvest_candidates`	L1752	tiny work-conservation harvest fallback
`endgame`	L1787	late or sparse-behind trigger
`YamoBot::commands`	L1793	complete per-turn policy driver
`Bot::commands`	L1893	controller interface contract
`main`	L1905	construct, read, decide, write, flush loop

55. Glossary

Term	Meaning in this manual
bank / inventory	resources already deposited at a shack; only these score
BFS	breadth-first search; exact shortest path on the walkable unweighted grid
candidate	one possible current command with a score and reservation target
capacity	maximum total cargo units a troll can hold
commitment	remembered seed-conversion intent for one troll and species
conversion	spend one banked fruit to plant, chop the new tree, and bank WOOD
cooldown	turns until a tree's next growth or fruit event
door	walkable grass orthogonally adjacent to a shack
endgame	turn >250, or ≤4 plants while our bank score trails
focus kind	permanent PLUM/LEMON species receiving enemy-shack denial bonus
free capacity	`carry_capacity - total_carried`
heuristic	a hand-designed estimate/rule used instead of exact future optimization
landing	cell reached this turn by a possibly farther MOVE target
opening	one-troll phase devoted to funding troll two
pair selection	exhaustive maximum over compatible candidate pairs for two trolls
reservation target	internal cell/tree/door identity used to prevent duplicate assignments
score band	large numeric range encoding priority between job classes
snapshot	complete referee state supplied at the start of one turn
throughput	collectible WOOD divided by full travel/chop/return/drop time
waypoint	rewritten one-turn MOVE landing, replanned next snapshot

56. Final mental model

When reading any function, ask four questions:

1. **What snapshot facts does it observe?** Positions, bank, cargo, skills, tree state, time.
2. **What alternatives does it create or remove?** Candidate generation and filters.
3. **What priority does it encode?** Score formula, score band, early return, or forced rewrite.
4. **What survives?** Usually nothing; only focus, training objective, opening state, announcement, and seed commitments cross turn boundaries.

If you can trace those four answers from `read_turn` through `commands` to `writeln!`, you understand the application. The rest is arithmetic, Rust syntax, and the consequences of doing short-horizon scheduling in a competitive world.